

Supplementary Material S13

HIMS-CDI Quick Start Guide for Clinical Safety Officers

Purpose

This guide provides a step-by-step procedure for a Clinical Safety Officer (CSO) to evaluate a candidate cybersecurity risk model using HIMS-CDI.

Prerequisites

- Python 3.11+ installed
- Access to the HIMS-CDI GitHub repository
git clone <https://github.com/oluseun-akeredolu/HIMS-CDI>
- Model outputs (predictions, latency, memory usage) in the required JSON format (see /schemas/model_output_schema.json)
- Clause mapping rules (provided in /clause_rules/)

Step 1: Install the HIMS-CDI package

```
bash
```

```
cd HIMS-CDI
```

```
pip install -r requirements.txt
```

```
python setup.py install
```

Step 2: Prepare your model's output file

Create a JSON file with the following structure:

```
json
```

```
{  
  "model_id": "your_model_v1",  
  "predictions": [0.82, 0.79, ...],  
  "ground_truth": [1, 0, ...],  
  "latency_ms": 145,  
  "memory_mb": 95,  
  "compliance_mappings": {  
    "GDPR-32-1-d": true,  
    "HIPAA-164-312-e-1": true,  
    ...  
  },  
  "explainability": {
```

```
"shap_available": true,  
"traceability_completeness": 0.9,  
"white_box": true  
}  
}
```

Step 3: Run the HIMS rubric evaluation

bash

```
python hims_rubric.py --input model_output.json --output hims_report.json
```

This produces:

- Structural Maturity Score (SMS) – range 0–18
- Pass/fail for qualitative gate (SMS ≥ 9 AND compliance ≥ 2 AND edge ≥ 2)

Step 4: Compute the CDI score

bash

```
python cdi_score.py --input model_output.json --weights ahp_weights.json --normalisation  
training_stats.json
```

Output:

CDI score: 0.77

Classification: DEPLOY (CDI ≥ 0.70)

Step 5: Apply governance thresholds

CDI range	Action
≥ 0.70	DEPLOY – automated edge integration under monitored conditions
0.40 – 0.69	CONDITIONAL – mandatory HITL review by CSO
< 0.40	REJECT – red team forensic audit required

Step 6: Generate Evidence Traceability Package (ETP)

bash

```
python generate_etp.py --input model_output.json --clause_rules clause_rules/ --output  
etp_audit.json
```

The ETP includes:

- Cryptographic hashes of evidence artefacts
- ISO 8601 timestamps
- Human-readable justifications

- Immutable audit log entry

Step 7: Archive for regulatory audit

Store the ETP JSON and any referenced artefacts in an immutable audit log (e.g., AWS S3 with object locking, or a blockchain-based notary). The SHA-256 hash of the ETP should be recorded in a separate, tamper-evident log.

Example command line (full pipeline)

bash

```
python hims_cdi_pipeline.py \
--model_output sample_output.json \
--weights weights/ahp_weights.json \
--norm_params training_stats.json \
--clause_rules clause_rules/ \
--output_dir ./results/
```

Troubleshooting

Issue	Suggestion
CDI < 0.70 but model is accurate?	Check latency and memory. If they exceed thresholds, consider model compression (quantisation, pruning) or switching to a lighter architecture.
Low compliance traceability?	Ensure your model outputs clause-level mappings. Use the <code>--force_mappings</code> flag to generate synthetic mappings for testing.
ETP validation fails?	Verify that all evidence artefacts are accessible and their SHA-256 hashes match.

Support

- Open an issue on GitHub: <https://github.com/oluseun-akeredolu/HIMS-CDI/issues>
- Email: Olu.Akeredolu@warwick.ac.uk (Corresponding author)